Ciências da Computação – 6º Semestre Noturno (NACCOMP6)

Entrega 02 – Teoria da Computação e Linguagens Formais

Sweet Spot

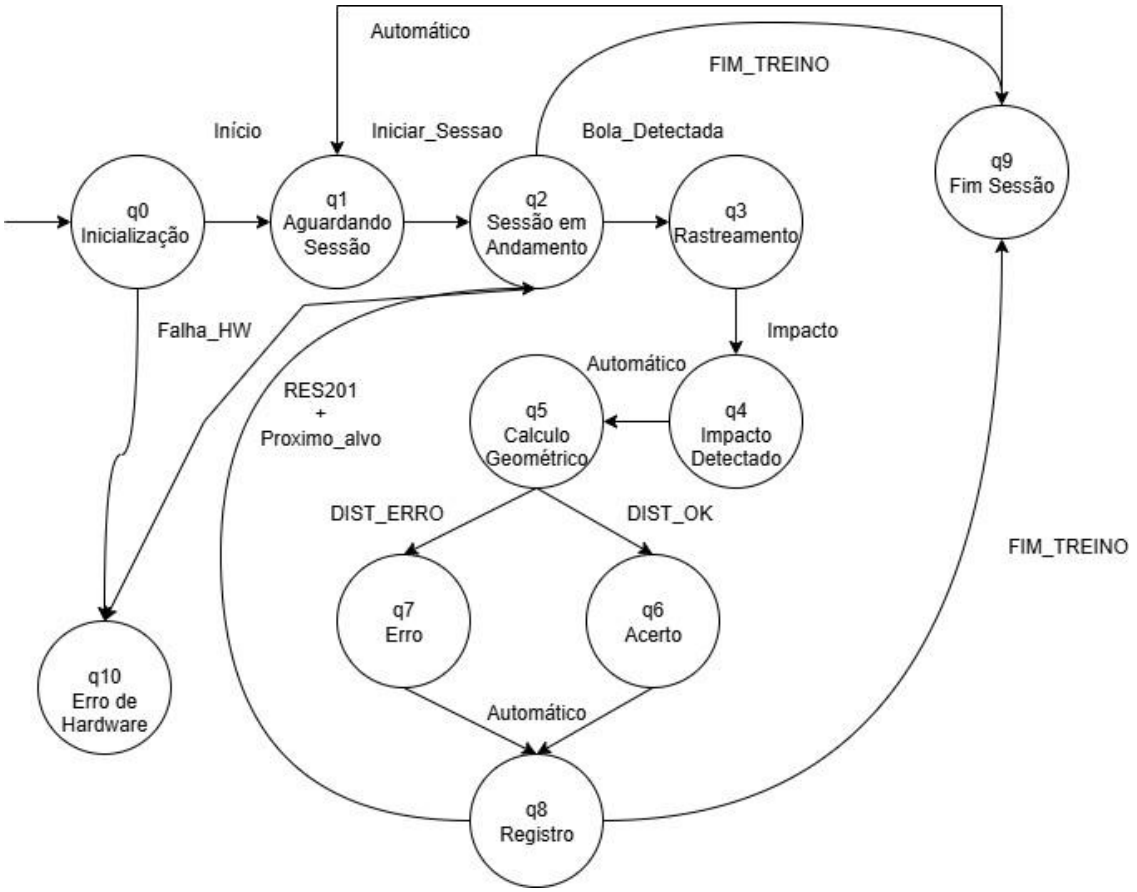
Arthur Paltrinieri (24026559)
Beatriz de Souza Santos Rio Branco (23025365)
Fabiano Henrique Chou (24025991)
Marcelo Henrique de Souza (23025665)

Resumo do projeto

O projeto Sweet Spot consiste numa solução tecnológica para o treino solo de precisão no tênis, integrando visão computacional, IoT e um sistema web gamificado. O sistema utiliza uma maquete com uma parede de LEDs controlada por Arduino, onde os LEDs atuam como alvos. Duas câmeras processam a imagem em tempo real através de um modelo YOLOv11 treinado (Label Studio), detectando o impacto de bolas de isopor por meio de heurísticas de inversão de trajetória, inversão de escala ou perda de rastreio. Os dados de precisão e tempo de reação são enviados para um backend Node.js/MySQL, que alimenta um dashboard interativo para análise de desempenho do jogador.

1. Atualização da máquina de estados

A estrutura geral da máquina de estados definida na Entrega 01 foi mantida, pois continua representando o ciclo principal de funcionamento do Sweet Spot. A revisão realizada nesta etapa corrige a ordem de duas transições para que o diagrama corresponda à descrição textual do projeto: a bola identificada leva o sistema de Q2 para Q3, e o gatilho de colisão leva de Q3 para Q4. Assim, o impacto não é processado antes do rastreamento. Os estados Q0 a Q10 permanecem os mesmos. Também foi explicitado o evento DIST_ERRO, já utilizado na tabela de transições da Entrega 01, como resultado complementar a DIST_OK: ocorre quando a distância calculada ultrapassa a tolerância estabelecida.



Obs: O evento FALHA_HW pode ocorrer em qualquer estado operacional do sistema (Q0 a Q9), direcionando a máquina para Q10 (Erro de Hardware). Por questões de legibilidade, as transições de falha não foram representadas individualmente no diagrama.

1.1. Descrição dos Estados

Estado	Descrição
Q0 – Inicialização	O sistema inicia os scripts de visão computacional, carrega o modelo YOLO, estabelece conexão Serial com o Arduino e aguarda a calibração da câmara.
Q1 – Aguardando Sessão	O hardware e o software estão prontos. O sistema aguarda o comando do frontend (via API) para iniciar um novo treino.
Q2 – Sessão em Andamento	O treino inicia. O Arduino acende um LED aleatório (alvo) e o sistema passa a monitorizar a área de treino.
Q3 – Rastreamento (Tracking)	O modelo YOLO está ativamente a rastrear a bola de isopor no *frame* da câmara em tempo real.
Q4 – Impacto Detectado	Uma heurística de colisão (inversão de trajetória, inversão de escala ou *motion blur*) identifica que a bola atingiu a parede.
Q5 – Cálculo Geométrico	O script calcula a distância em centímetros entre a coordenada (X,Y) do impacto detectado e o centro do LED ativo.
Q6 – Acerto (HIT)	A distância calculada é menor ou igual à tolerância definida (ex: 10 cm).
Q7 – Erro (MISS)	A distância calculada é superior à tolerância definida, ou a bola atingiu a parede fora da zona ativa.
Q8 – Registro (Telemetria)	Os dados da jogada (acerto/erro, precisão em %, tempo de reação) são enviados via requisição HTTP (POST) para a API Node.js.
Q9 – Fim de Sessão	O tempo do treino esgotou ou o limite de batidas foi atingido. Os dados finais são consolidados na base de dados MySQL.
Q10 – Erro de Hardware	Perda de conexão com as câmaras ou com a porta Serial do Arduino.

1.2. Tabela de transições

Estado Atual	Evento (Entrada)	Próximo Estado
Q0 (Inicialização)	Inicio	Q1 (Aguardando Sessão)
Q0 (Inicialização)	FALHA_HW	Q10 (Erro de Hardware)
Q1 (Aguardando Sessão)	Iniciar_Sessao	Q2 (Sessão em Andamento)
Q2 (Sessão em Andamento)	Bola_Detectada	Q3 (Rastreamento)
Q2 (Sessão em Andamento)	Fim_Treino	Q9 (Fim de Sessão)
Q3 (Rastreamento)	Impacto	Q4 (Impacto Detectado)
Q4 (Impacto Detectado)	(Automático)	Q5 (Cálculo Geométrico)
Q5 (Cálculo Geométrico)	DIST_OK	Q6 (Acerto)
Q5 (Cálculo Geométrico)	DIST_ERRO	Q7 (Erro)
Q6 (Acerto)	(Automático)	Q8 (Registro)
Q7 (Erro)	(Automático)	Q8 (Registro)
Q8 (Registro)	RES201 + Proximo_Alvo	Q2 (Sessão em Andamento)
Q8 (Registro)	RES201 + Fim_Treino	Q9 (Fim de Sessão)
Q9 (Fim de Sessão)	(Automático)	Q1 (Aguardando Sessão)
Qualquer Estado	FALHA_HW	Q10 (Erro de Hardware)

2. Descrição dos eventos

Os eventos representam as condições que provocam a mudança de um estado para outro dentro do sistema. Eles podem ser originados por comandos da aplicação, resultados do processamento de visão computacional, respostas da API ou falhas nos componentes físicos. Os eventos definidos em nosso projeto são os seguintes:

Evento	Símbolo	Descrição
Configuração Concluída	Início	Calibração e conexão com hardware concluídas.
Comando de Início	Iniciar_Sessao	O usuário clica em "Nova Sessão" no *dashboard* web.
Bola no Frame	Bola_Detectada	O YOLO identifica o objeto "bola" na imagem.
Gatilho de Colisão	Impacto	Mudança abrupta nos vetores de movimento ou perda de rastreio na zona calibrada.
Distância ≤ Tolerância	Dist_OK	Impacto próximo o suficiente do LED aceso.
Distância > Tolerância	DIST_ERRO	Indica que a distância calculada ultrapassou a tolerância definida ou que o impacto ocorreu fora da região válida.
Confirmação de API	RES201	O backend confirma a gravação da batida no banco de dados.
Novo Alvo Gerado	Proximo_Alvo	O Arduino recebe comando para acender outro LED.
Condição de Parada	Fim_Treino	O limite de batidas (ex: 10) ou o tempo da sessão foi atingido.
Desconexão	FALHA_HW	Cabo USB desligado ou falha de leitura de vídeo/serial.

3. Regras de funcionamento

O funcionamento do Sweet Spot depende de um conjunto de regras que determinam como o sistema deve se comportar durante uma sessão de treinamento.

Regra 1 – Inicialização dos componentes: o sistema somente poderá iniciar uma sessão após concluir o carregamento do modelo YOLOv11, a calibração das câmeras e a comunicação serial com o Arduino.

Regra 2 – Início da sessão: uma nova sessão somente poderá ser iniciada quando o sistema estiver no estado Q1, aguardando o comando da aplicação.

Regra 3 – Seleção do alvo: ao iniciar uma sessão, o Arduino deverá acionar um LED aleatório, que representará o objetivo da jogada.

Regra 4 – Detecção da bola: o sistema deverá utilizar o modelo YOLOv11 para identificar a bola de treinamento dentro da área monitorada.

Regra 5 – Rastreamento: após a identificação da bola, o sistema deverá acompanhar seu deslocamento até identificar uma possível colisão com a superfície.

Regra 6 – Identificação do impacto: o impacto deverá ser identificado por meio das informações de trajetória e das heurísticas implementadas no processamento de visão computacional.

Regra 7 – Cálculo de precisão: após a identificação do impacto, o sistema deverá calcular a distância entre o ponto de colisão e o centro do LED ativo.

Regra 8 – Classificação da jogada: a jogada será classificada como HIT quando a distância calculada for menor ou igual à tolerância estabelecida. Caso contrário, será classificada como MISS.

Regra 9 – Registro dos resultados: todas as jogadas classificadas deverão ser encaminhadas para a API, que será responsável pelo armazenamento das informações no banco de dados MySQL.

Regra 10 – Confirmação do registro: o sistema deverá aguardar a confirmação da API antes de prosseguir para a próxima jogada.

Regra 11 – Encerramento da sessão: o treinamento deverá ser encerrado quando o tempo disponível terminar ou quando o limite de jogadas for atingido.

Regra 12 – Tratamento de falhas: caso ocorra uma falha de comunicação com as câmeras ou com o Arduino, o sistema deverá interromper o fluxo normal e entrar no estado Q10, destinado ao tratamento de erros de hardware.

4. Pré-condições e Pós-condições das Principais Operações

A definição das condições de pré-condições e pós-condições permite estabelecer critérios objetivos para validar as operações realizadas pelo sistema.

Operação	Pré-Condições	Pós Condições
Inicialização	Componentes físicos conectados e disponíveis.	Modelo carregado, câmeras calibradas e comunicação estabelecida.
Início da sessão	Sistema no estado Q1 e componentes operacionais.	Sessão iniciada e primeiro alvo selecionado.
Detecção da bola	Sessão em andamento e câmeras funcionando.	Bola identificada e rastreamento iniciado.
Detecção do impacto	Bola em processo de rastreamento.	Impacto identificado e coordenadas disponíveis.
Cálculo geométrico	Coordenadas do impacto e centro do LED conhecidos.	Distância calculada e resultado classificado.
Registro da jogada	Resultado da jogada disponível e API acessível.	Informações encaminhadas e confirmação de registro recebida.
Geração de novo alvo	Registro anterior confirmado e sessão ainda ativa.	Novo LED selecionado e próxima jogada iniciada.
Encerramento da sessão	Tempo esgotado ou limite de jogadas atingido.	Sessão encerrada e dados consolidados.

Essas condições garantem que cada operação seja executada dentro de uma sequência lógica, evitando que o sistema realize cálculos sem informações suficientes ou registre resultados incompletos.

5. Justificativa do Modelo Computacional Escolhido

Para o Sweet Spot, o modelo que optamos foi o de Autômato Finito, pois o funcionamento principal do sistema pode ser representado por um conjunto limitado de estados, nos quais cada transição depende do estado atual e da ocorrência de determinado evento.

O sistema possui etapas bem definidas, como inicialização, espera por sessão, rastreamento da bola, identificação do impacto, cálculo geométrico, classificação do resultado e registro da telemetria.

Não existe a necessidade de uma pilha para controlar estruturas aninhadas, tampouco de uma fita de memória ilimitada para representar o fluxo principal de execução. As informações adicionais, como coordenadas, distâncias e resultados das jogadas, são processadas pelos componentes internos do software e não alteram a natureza finita da máquina de estados utilizada para representar o controle da sessão.

Portanto, o Autômato Finito apresenta a complexidade necessária para representar o comportamento operacional do Sweet Spot, sem exigir recursos computacionais mais abrangentes para essa finalidade.

6. Análise de decibilidade

A decidibilidade está relacionada à possibilidade de determinar, por meio de um procedimento computacional, se determinada condição ou problema possui uma resposta definida.

No Sweet Spot, diversas decisões são realizadas durante a execução do sistema. Essas decisões são baseadas em condições objetivas, permitindo que o software determine de maneira sistemática qual ação deverá ser executada.

Um exemplo é a validação do comando de início da sessão. O sistema verifica se está no estado Q1 antes de aceitar o evento INICIAR_SESSAO. Caso esteja em outro estado, o comando não deverá provocar a transição para Q2.

Outro exemplo é a classificação da precisão da jogada. Após calcular a distância entre o impacto e o centro do LED ativo, o sistema compara o resultado com a tolerância estabelecida. Se a distância for menor ou igual ao limite, a jogada será classificada como HIT. Caso contrário, será classificada como MISS.

Também é possível determinar se uma sessão deve ser encerrada. Para isso, o sistema verifica se o tempo disponível terminou ou se o limite de jogadas foi atingido. Como essas condições podem ser representadas por comparações objetivas, o resultado da verificação é determinístico.

A confirmação do registro também constitui uma decisão verificável. O sistema analisa o código de resposta recebido da API e determina se o registro foi confirmado por meio da resposta HTTP 201.

Por fim, o sistema verifica a disponibilidade dos componentes físicos, como câmeras e Arduino. Caso identifique uma falha de comunicação, deverá executar a transição correspondente para o estado Q10.

Exemplos de decisões realizadas pelo sistema:

Decisão	Condição analisada	Resultado possível
Validação do início	Estado atual é Q1?	Aceitar ou rejeitar o comando.
Classificação da jogada	Distância menor ou igual à tolerância?	HIT ou MISS.
Encerramento da sessão	Tempo esgotado ou limite de jogadas atingido?	Continuar ou encerrar.
Confirmação da API	Resposta HTTP 201 recebida?	Prosseguir ou tratar falha no registro.
Verificação de hardware	Câmeras e Arduino estão disponíveis?	Continuar operação ou entrar em Q10.

Essas decisões são consideradas decidíveis porque podem ser resolvidas por procedimentos computacionais finitos, baseados em comparações e verificações de condições previamente estabelecidas.

7. Considerações finais

A segunda etapa do projeto Sweet Spot permitiu aprofundar a representação formal do funcionamento do sistema, estabelecendo regras operacionais, condições para execução das principais operações e critérios para a tomada de decisões.

A atualização da máquina de estados possibilitou organizar melhor o fluxo de execução, desde a inicialização dos componentes até o encerramento da sessão de treinamento. Além disso, a definição dos eventos e das pré-condições e pós-condições contribuiu para estabelecer um comportamento mais consistente e previsível.

A escolha do Autômato Finito como modelo computacional foi justificada pela natureza do fluxo de execução do sistema, que apresenta estados limitados e transições condicionadas por eventos específicos.

Por fim, a análise de decidibilidade demonstrou que as principais decisões realizadas pelo Sweet Spot podem ser resolvidas por procedimentos computacionais finitos, permitindo que o sistema determine de maneira objetiva suas transições e ações.

Dessa forma, a Entrega 02 complementa a modelagem apresentada na primeira etapa, consolidando a especificação formal do comportamento esperado do projeto.